[A New Player] Has Joined The Game
A Decentralised Topology API

A thesis submitted for the degree of
BA (Hons) Game Design and Production

by

Louis Baker

Faculty of Design, Informatics and Business,
Abertay University.

March, 2025

Declaration

Candidate’s declarations:

I, Louis Baker, hereby certify that this dissertation submitted in partial fulfilment of the requirements for the award of BA (Hons) Game Design and Production, Abertay University, is wholly my own work unless otherwise referenced or acknowledged. This work has not been submitted for any other qualification at any other academic institution.

Signed Louis……
Date 05/04/2025…….

Acknowledgements

I would like to express my deepest gratitude to my supervisor Dr William Kavanagh for their guidance, feedback, and support throughout this project. Their insights have been instrumental in shaping this project and dissertation. I would also like to thank the module tutors: Prof Robin Sloan, Dr Mona Bozdog, and Dr Ege Sezen for their feedback throughout the module, always lending a hand when needed. Additionally, I am grateful for my friends and peers that I have met throughout the duration of this course. Without them this project would not have been possible. Finally, I am thankful for Abertay for providing me with the opportunity to pursue my interests in game development through this course, and all the lecturers who have helped guide me to where I am now.

Abstract

Developing Multiplayer games as an indie developer can be challenging. There are various available tools that each offer unique features, but they are based on a client-server architecture. This issue is tackled by the proposed API that utilizes a decentralized topology. The literature review on how latency can impact player enjoyment provides an overview of latency's impacts on the UX and highlights techniques that can be used to mitigate them. The field study analyses two available APIs for indie developers, summarizing their core features and outlining the benefits and drawbacks of the client-server architecture. This study helps inform the development of the proposed API. The dissertation evaluates the implementation of this API in the development of a demo game. The results demonstrate the APIs potential for multiplayer development and identify the differences between a centralized and decentralized topology.

Table of Contents

List of Figures

Definitions

API – Application Programming Interface function as intermediaries, enabling software components to interact and exchange information without needing to know the internal workings of the other application, in this paper it specifically refers to network APIs.

Topology – refers to network topologies which is the physical and logical arrangement of nodes and connections within a computer network.

Decentralized – refers to a network architecture where control and decision-making are distributed across multiple nodes or participants, rather than being concentrated in a single entity or serve.

P2P – Peer-to-Peer architecture: each participant in a network is both a client and a server, meaning they can request resources and share their own.

CS – Client-Server architecture: the server hosts, delivers, and manages most of the resources and services requested by the client.

1. Introduction

This dissertation investigates the validity of a decentralized topology API for multiplayer game development. As most network solutions available to indie developers use a Client-Server model, this model challenges the status quo to determine whether it is comparable to, or even better than, a Client-Server model for multiplayer development.

1.1 Motivations

My personal motivation for this dissertation was to develop and evaluate a network solution made explicitly for a decentralized network topology. This was due to the lack of existing literature on this network topology as well as available solutions that are specifically designed for it. In the dissertation I examine my alternate solution: reducing network latency's impacts on player enjoyment and democratizing development tools through my own networking solution. The results inform developers of alternate solutions and provide a prototype API to adapt or use.

1.2 Research Questions

RQ 1	**How does network performance impact player enjoyment?**
RQ 2	**What are the differences between centralised cand decentralized network topologies?**
RQ 3	**What are the difficulties when designing and programming a game using a decentralized network topology?**

1.3 Dissertation Structure

The dissertation includes a Literature addressing RQ 1, it also provides validity for my proposed model. Followed by a field survey comparing two existing network solutions available for Unity: Fish Networking and Photon which helps inform the practical methodology, answering RQ 2. An application of this practical methodology to develop a network solution. The evaluation of this solution and the conclusion addressing RQ 3.

2. How does network performance impact player enjoyment?

Network performance has always played a role in game design ever since the release of the first multiplayer games. Its impact on user experience has always been a prevalent issue, with it being "one of the largest sources of frustration for online gamers" (Madanapalli, Gharakheili, & Sivaraman, 2022). Although the phrase *network performance* is somewhat vague, it can be broken down into more nuanced topics of which, three will be discussed in this review: contextual factors such as game mode or genre; latency mitigation methods like interpolation or prediction; and variables within the network, including network topology, server and client locations, and security. The topics discussed here do not fully encompass everything that can influence user experience, but they are still key to understanding how to address issues related to *network performance*.

2.1 Do Contextual Factors Significantly Influence the Effects of Latency on the User Experience?

2.1.1 Latency affecting player performance

The relationship between a user's experience (UX) and network performance is complex, many external factors can affect any one experience within a game causing results to be invalid or inconclusive, as outlined in Vlahovic's, et al. paper where they found that "latency above 100 ms had a noticeable negative impact" on the player's experience however then stating that "contextual factors have a significant influence on subjective metrics" (Vlahovic, Suznjevic, & Skorin-Kapov, 2019). When comparing the results of their own studies, which could undermine the validity of the results.

However, these factors appear to be mitigated when comparing multiple studies' results, as done in Claypool & Claypool, (2006) paper: *Latency and player actions in online games* where individually studies may consider the context and additional factors when drawing their own conclusion, Claypool & Claypool draw from the similarities across

each study whilst maintaining some level of context such as game types.

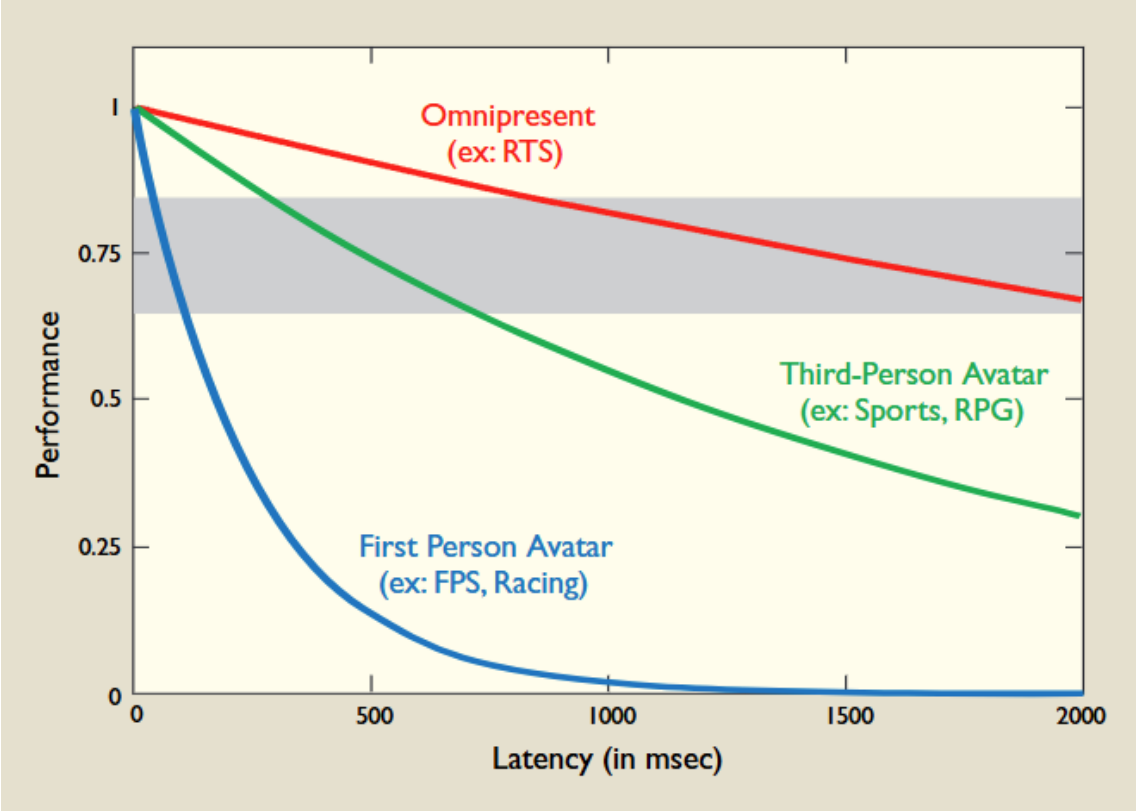

Figure 1 - Player performance versus latency for game categories. The horizontal gray band represents the threshold for player tolerance for latency

This graph shows that first-person avatar games, such as FPS and racing, show users' performance being rapidly influenced by latency (0ms - ~250ms). Whereas Third-Person avatar games, such as Sports and RPG, show a more lenient range of latency affecting performance (0ms - ~700ms), with Omnipresent games, such as RTS, having latency's influence having the least impact on performance (0ms - ~2000ms) (Claypool & Claypool, 2006).

2.1.2 Network Loss affecting player performance

Although Claypool & Claypool conclusively find a relation between latency and performance, network loss also plays a significant role as outlined in *On the Sensitivity of Online Game Playing Time to Network QoS:* "network loss has a much higher impact than latency" (Chen, Huang, Wang, Huang, & Lei, 2006) and comparing the influence of network loss and network latency they found a ratio of 17:3 (Chen, Huang, Wang, Huang, & Lei, 2006). This is further backed by García Fernández de Castro & Jabba Molinares where they state that a network "susceptible to packet loss" is "unsuitable in the context of video games." (García Fernández de Castro & Jabba Molinares, 2023).

But when comparing their results to an earlier study on *Unreal Tournament 2003* (Beigbeder, et al., 2004) they state: “though network latency in a typical range (0ms – 200 ms) has a statistically weak impact on user performance, network loss of a typical range (<6%) has no impact on user performance.” (Chen, Huang, Wang, Huang, & Lei, 2006).

Harcsik et al. express similar findings, though they note that their figures worst-case latency values show large variation on the effect on game playability (Harcsik, Petlund, Griwodz, & Halvorsen, 2007). This network loss can have a significant impact on the Quality of Service (QoS), as mentioned in García Fernández de Castro & Jabba Molinares paper: it is “crucial in online multiplayer games as it can significantly affect players' experiences” (García Fernández de Castro & Jabba Molinares, 2023). If there is a degridation in QoS, it could negatively effect the user experience.

Overall, contextual factors play a significant role when studying the impact of network performance on the UX, especially towards the higher values of latency, with the only notable difference being the latency tolerance across different game types.

2.2 How Can Latency Mitigation Methods Effect User Experience?

2.2.1 Complex Lag Compensation Techniques

In the pursuit of a better gameplay experience, developers have implemented latency compensation methods to minimize the effects of network variance. Savery and Graham state: “Advanced lag compensation techniques show great promise for improving user experience,” evidenced using ‘remote lag’ in the Half-Life games which improves both player experience and performance (Savery & Graham, 2013). However they mention the limitations of these methods being their need for evaluation within each specific game, they are also further limited by their difficulty to implement as if too “cumbersome” there is a significant barrier to even evaluate them (Savery & Graham, 2013).

Although these methods may not always yield positive results as some delay compensation techniques, if over tuned, can cause a mundane gameplay experience (Sabet, 2023), since “the existence of delay makes the game more challenging and

adaptation should be employed to balance the intensity of the challenge", (Sabet, 2023). Sabet only validated the effects of delay within singleplayer games which limits their findings, although they close stating that these methods could significantly improve the gaming QoE when there is variation in delay, resonating with the goals of lag compensation methods which aim to minimize the effect of network variance by making gameplay consistent.

2.2.2 Simple Lag Compensation Techniques

Whereas Sabet looked at more complex methods applied within a singleplayer game, Bernier analysis earlier compensation methods such as interpolation and extrapolation. Extrapolation is when a player/object is simulated forward in time from their last know spot (Bernier, 2003), but due to the "not very ballistic, but instead non-deterministic" movement of players this often results in the player's perceived actions being incorrect to their actual actions. Conversely, interpolation can be viewed as always moving objects in the past with respect to the last valid position received for the object. Interpolation helps smooth the movement of players/objects rather than teleport the player to their most recent position, "resulting in a more responsive gaming experience" (Spina, et al., 2024).

In the scenario where a packet does not arrive the client can decide to extrapolate or wait for a new packet, due to the problems that extrapolating can cause, it is often best to wait although there are always niche scenarios. Except interpolation is not always a bullet proof method, it often "flattens" movement discrepancies due to a longer delay between packets, this can result in movement not accurately reflecting what is happening on either server or client. This can be mitigated by keeping a more complete history of earlier positions, but this may only succeed for predicable movements. They conclude that the decision to implement these tools is up to the designer, as it can impact game feel differently depending on the game/genre, but for their discussed games (Half-Life, Team Fortress, Counter Strike) the "benefits of lag compensation easily outweighed the inconsistencies" (Bernier, 2003).

However, with simpler lag compensation techniques they aim only to mitigate the effects of latency for the client with latency. This can result in unaffected clients receiving inconsistent data from on client which may degrade the experience (Spina, et

al., 2024). This simpler model also “opens up more opportunities for cheating, as the client has greater control over the game state it reports” (Spina, et al., 2024).

Most scholars encourage the use of latency mitigation techniques but with a heavy emphasis on testing for each game as the effects of them can vary case to case and may not always yield a positive impact.

2.3 How Do Variables Within a Network Effect the User Experience?

2.3.1 Physical Location of Servers

Variables within a network are a deciding factor in whether developers can program and implement certain methods or techniques to mitigate the effects of latency, but they may also have a direct correlation to the user experience. “Real-time gaming experience is heavily influenced by factors such as the distance between servers and gamers” (Hami, Boujelben, & Zarai, 2024). To provide a fair experience to all connected clients servers can utilise a synchronization delay where the server will wait, based on the client with the greatest delay, before performing calculations (Lee, Ko, & Calo, 2005). This delay is influenced by the physical location of clients as well as the physical location of the server. The shorter the distance the data must travel results in lower latency, thereby improving network speed which is critical for maintaining high-quality gaming experiences (Hami, Boujelben, & Zarai, 2024). If a server is positioned poorly then clients will have a higher latency on average which will negatively impact the user experience.

2.3.2 Peer-to-Peer Model’s Strengths and Weaknesses

For Peer-to-Peer architecture, the age of empires genre provides a solid example of the strengths and weaknesses it offers which Bettner and Terrano list in their paper *“Network Programming in Age of Empires and Beyond”* (Bettner & Terrano, 2001). They found that P2P has a reduced latency overall providing a more consistent connection. The architecture also has no single of point of failure (Bettner & Terrano, 2001) (Bamutange, Ramsurrun, Seeam, Katsina, & Anantwar, 2020), which provides a less volatile connection for each peer as in the event they lose synchronisation with another peer, all other peers can quickly help correct them. The Peer-to-Peer architecture also has the ability to scale depending on the number of active users since

each peer is a server and client at the same time (Bamutange, Ramsurrun, Seeam, Katsina, & Anantwar, 2020).

The security of P2P was superior to current client-server models as a single cheating peer had to convince all other peers to cheat successfully which was impossible (Bettner & Terrano, 2001). Peers could still reveal information locally, on their own system, within early versions but this was later patched. Bettner and Terrano state that because of the RTS genre, P2P works for Age of Empires, yet they emphasise on knowing your player to adjust network parameters for a smooth gameplay experience (Bettner & Terrano, 2001). However, the abundance of dedicated servers makes this architecture less popular today (Bamutange, Ramsurrun, Seeam, Katsina, & Anantwar, 2020).

2.3.3 Client-Server and Peer-to-Peer Hybrid

The Client-Server model has benefits such as having a single centralised server acting as the authority making it easier to manage and secure data transmitted (Bamutange, Ramsurrun, Seeam, Katsina, & Anantwar, 2020). This makes the CS model popular with game that have a relatively small number of concurrent players at a given time (Bamutange, Ramsurrun, Seeam, Katsina, & Anantwar, 2020). But in Massive Multiplayer Online Games, the standard CS model experiences limitations related to latency, reliability, scalability (Bamutange, Ramsurrun, Seeam, Katsina, & Anantwar, 2020) (Pellegrino & Dovrolis, 2003).

Pellegrino and Dovrolis go on to propose their own architecture that makes use of both peer-to-peer (P2P), and client-server (CS) architectures dubbed “Peer-to-Peer with Central Arbiter, or PP-CA, (Pellegrino & Dovrolis, 2003). They conclude that their model achieves a low latency between players inherited from the P2P aspect of their model but with the ability to resolve inconsistencies easily through the central arbiter which bypasses the need for a complex agreement amongst peers present in the P2P model. However, the central arbiter does not have a reduced processing load, as it still simulates the state of the entire game just like a CS architecture. (Pellegrino & Dovrolis, 2003). This could lead to a delay not caused by latency between clients and the central arbiter but the instead the processing time required before the central arbiter can send out the resolved game state.

Bamutange, et al. instead propose a hybrid CS model to be used for MMOGs where servers are split into clusters that dedicated processing power and bandwidth to *areas of interest* (Bamutange, Ramsurrun, Seeam, Katsina, & Anantwar, 2020). These areas of interest can be caued by a build up of players in a specific part of the game world, the servers that have the least amount of players within a cluster can then dedicate themselves to handling traffic from the more busy servers to alleviate the load. However, this model is “costly to implement” and “bandwidth limiations for servers hinders scalability” (Bamutange, Ramsurrun, Seeam, Katsina, & Anantwar, 2020).

To summarise, network variables can significantly impact the player experience not only directly but also through their influence on what developers can do to mitigate the effects of latency due to limitations, as well as what clients can do within the network whether it be manipulating data to gain an advantage or increase the overall delay for all clients due to their individual latency.

2.4 Conclusion

Many papers study various aspects that can affect the user experience. Ranging from contextual factors that can vary from case to case, to frameworks that can enable developers to implement lag compensation techniques or prevent the use of certain techniques. Most of the scholars encourage all developers to test and verify the validity of their proposed methods before committing to using them as the resulting impact on game feel can vary drastically.

Higher latencies between clients and servers can have their effects mitigated using a tailored lag compensation technique which will help diminish the influence contextual factors may have on the player experience, but the network variables may dictate what can or cannot be implemented.

Each of the discussed topics can have a noticeable impact on a user’s experience in varying ways, and therefore player enjoyment (RQ 1). These individual topics can be managed in isolation to alleviate bad experiences. Although it is best to manage each of the topics with respect to one another as this can create a more enjoyable user experience.

3. Analysis of Existing Network Solutions

3.1 Fish-Net API

Fish Networking is a networking solution aimed towards reliability, ease of use, efficiency, and flexibility. One prominent feature it boasts over others is its long-term support, constantly keeping the source code up to date with bug fixes and improvements, with a No-Break Promise (FirstGearGames, 2022). Many developers can use this solution for their games due to its breadth of features.

3.1.1 Feature List

Robust Remote Calls

Fish-Net's Remote Calls are tweaked to allow them to be run on their intended destination but also the caller as well. This helps reduce code bloat since the user does not have to copy logic into multiple methods which are typical of net code. This aids in reducing potential errors and improves ease of use. Their remote calls are also designed to be multi-purpose, letting users send data to one or multiple clients through a single method improving the user experience and code reliability.

Network Managers

Network managers sit at the core of net code, handling communication between clients and servers. FishNet allows for an infinite number of network managers, enabling clients to connect to more than one server and handle various tasks. This allows users to split up elements of their game to be managed by individual managers such as game modes or transports.

Lag Compensation and Prediction

They offer client-side prediction, which can prevent players from altering their movement to cheat helping combat malicious players from ruining everyone's experience. This feature also helps smooth movement caused by desynchronization helping reduce the effects of latency.

Although a paid feature, FishNet offers lag compensation net code for developers. By rolling back colliders to when a client would have seen them to ensure accurate hit

registration, to help improve game feel and mitigate the effects of latency. This technique applies to multiple game genres but primarily for shooters.

Automatic Code Stripping

A paid feature, FishNet offers code stripping which helps to protect game servers by removing sensitive game logic that could otherwise be used to gain an unfair advantage. This feature also helps combat the effects of latency players can experience such as rubber banding.

3.1.2 Conclusion

Only a few of the unique features are covered here, FishNet has even more available and has improved upon common features as well. Having such breadth and depth of features lets the API be flexible and fit many different game and network types. The long-term support keeps the solution up to date whilst supporting any older code not requiring developers to update their net code alongside them. Being a free solution, FishNet sits at the pinnacle of available APIs for Unity.

3.2 Photon API

Photon offers both free and paid networking solutions, with their free solution functioning like a demo still allowing users to develop functioning multiplayer but with limited features. The solution limits concurrent users on the network which can be increased through paid tiers, but the highest free tier is one hundred concurrent users at a time. The key features available appear to be oriented towards competitive games.

3.2.1 Feature List

Lag Compensation and Prediction

Like FishNet, Photon offers lag compensation to alleviate the effects of latency by rolling back hitboxes to validate potential hits. Also offering client-side prediction to have instant response to player input even on a server authoritative model which reduces the effect of latency for a player’s movement.

No Net Code Programming

Photon's paid product: Quantum allows developers to implement multiplayer functionality into their games without touching any net code by letting developers create a local multiplayer experience that does not require any network interaction, quantum then assumes any communication between players to be networked. This helps developers swiftly design multiplayer games without requiring knowledge of how networks interact with each other.

Cheat Protection

Photon Quantum offers numerous ways to combat cheaters and explains their drawbacks to inform developers on their design decisions. One method is protection by determinism, as any player that tries to cheat by modifying aspects such as their speed will not affect other players. This is due to Quantum's design as it only sends user input to other clients, which these clients use to simulate the actions of that user.

Quantum also offers a *referee simulation* to validate user actions on a separate instance which further protects against cheaters in case the client's simulation does not. This offers an alternative protection method for developers that does not require any extra programming from the developer, this flexibility offered allows developers to focus on making the game their way without worrying about cheat protection.

3.2.2 Conclusion

Photon's features and design offers developers an easy to implement multiplayer solution without the need for complex net code programming, or any at all for Quantum. The solution has more features available than is covered here, but these are the core features that they advertise for both their free and paid versions. Overall, the solution lets developers implement multiplayer very quickly and easily, with lag compensation and cheat protection. The draw back to their design is it cannot be easily altered if developers need a more tailored solution for their game.

3.3 Comparison

Both network solutions share many similarities, especially with their features however there a few notable differences.

Feature	Fish Networking	Photon
Pricing	Free (contains paid features).	Partially Free (offers limited concurrent users for development and launch, then requires payment).
Hosting	Has an official partner that offers hosting as well as other recommended providers.	Offers their own hosting.
Cheat Protection	Has no dedicated cheat protection available.	Has its own cheat protection system: *referee simulation.*
Matchmaking	Does not have matchmaking but has lobby systems in place for it.	Offers their own matchmaking system.
Addons	Source code is publicly available allowing users to view and modify it, a large number of addons have already been created and are publicly available.	Offers their own addons , source code is not publicly available therefore no community generated addons.

4. Methodology

4.1 Common Network Topologies

4.1.1 Client-Server Model

The most common architecture for multiplayer gamers is the Client-Server model. In this architecture the server has the authority and clients send their actions to the server to be processed, with the results then being sent back to the clients. This model offers more security and control over the game environment but at the cost of high network latency which negatively affects the user experience as outlined previously. Client-

Server models have higher running costs due to the entire simulation needing to be processed on a separate server which poses a problem for smaller studios as their budget is not as substantial as larger studios.

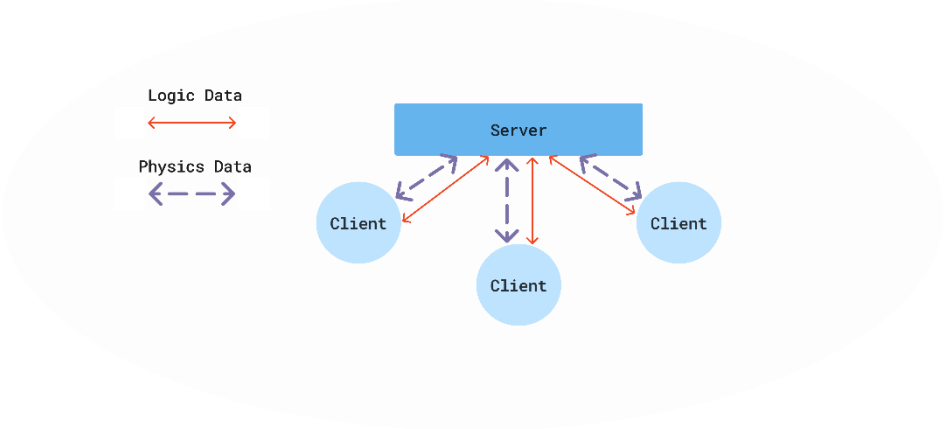

Figure 2 - Client-Server Network Topology

4.1.2 Peer-to-Peer Model

The alternative to a client-server model is a peer-to-peer model. This model does not utilise a central server but instead has all clients connect to each other. This allows removes the need for dedicated servers making this topology cheaper for developers but, the immediate drawback is the lack of security and control as each peer has authority over their own actions, allowing them to cheat more easily. This problem can be mitigated somewhat with peers validating each other's actions however cheating peers will always have access to all information happening within the game, which can be used to gain an advantage.

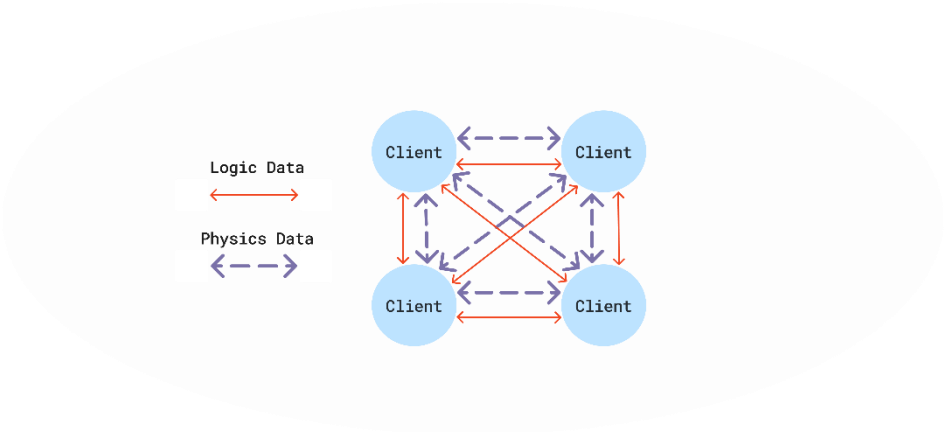

Figure 3 - Peer-to-Peer Network Topology

4.2 Development of a Decentralized Topology API

4.2.1 Summary

The architecture I have developed uses a mix between Client-Server and Peer-to-Peer architecture where game physics is calculated on a host peer's device and game logic is managed by a separate server. This model allows developers to maintain security over critical components such as game state or player progression whilst delegating expensive physics calculations to a host peer. The model is also client authoritative which removes the effects of latency for local actions such as moving and shooting, helping to improve the user experience.

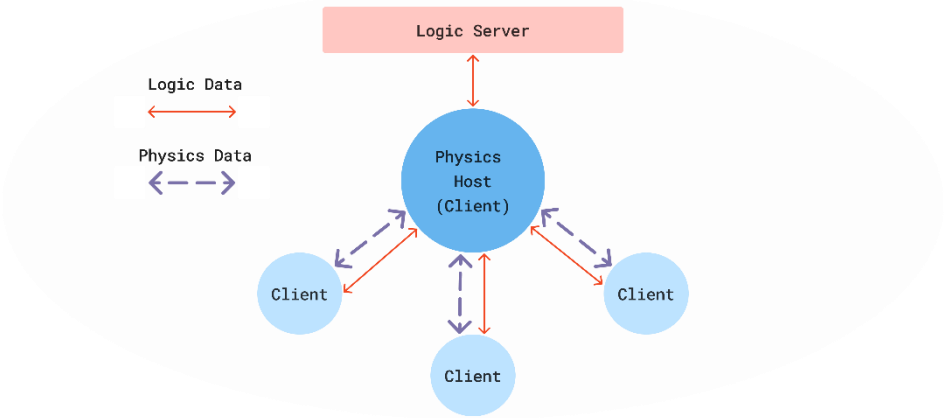

Figure 4 - Decentralized Network Topology

4.2.2 Features

Client Authoritative

The model uses client authority which means clients do not have to make requests to the server when performing local actions such as moving or shooting. This removes the effects of latency in relation to a player's local actions, the most common one being rubber banding, where a player is teleported to an earlier position because of the server's and client's account of their position being desynchronized. Client authority does have the disadvantage of being less secure, as clients can alter their local variables such as move speed, meaning cheaters can more easily gain an advantage.

Split workload

The Physics calculations are handled by a host peer allowing the separate logic server to have cheaper running costs due to its reduced workload, or it can delegate spare processing power to extra features such as matchmaking services. The physics calculations being handled by a peer does create security concerns as this peer does have access to all information about the simulation, this could be managed by an external referee to ensure that no clients behave unexpectedly.

4.3 Case Study

4.3.1 Overview

To evaluate the proposed model, I developed a small demo game. The demo game is a short two player cooperative experience where players would traverser through a dungeon with AI enemies and eventually fight and defeat boss at the end of the dungeon. Crucial game logic would be handled via an external server that the host client would communicate with; the host client would relay logic information to other connected clients when necessary. Throughout the experience I would simulate network fluctuations such as latency to showcase the strengths of a decentralized topology.

4.3.2 Design

The demo game was split between networking systems and gameplay systems. These two systems would communicate with each other when necessary, allowing them to run independently from each other which prevented errors from crashing the entire game.

4.3.2.1 Networked System

The networked systems used the scripts provided by the API to communicate across the network. The main classes that were edited and used were the send and receive classes. This would predominantly be physics related functions such as moving, shooting, or spawning. With the remaining functions being related to the game logic which was sector doors opening once certain enemies had been destroyed.

The logic server also predominantly used the scripts provided by the API, again with the send and receive classes being predominantly edited. The logic server would only connect to the host client, sending logic information to the host client such as state changes to allow clients to progress within the game.

4.3.2.2 Gameplay System

The gameplay system consisted of the character controller, game manager, and the enemy manager. The character controller was a simple script allowing clients to move their characters within a 3d space as well as interact with it by shooting or getting shot. These scripts would call networked function when necessary, such as constantly updating their position and rotation or shooting so other clients could see each other's actions.

The game manager would spawn in and keep track of players with a dictionary which gave players an identifier, which meant clients could reference each other over the network an know which player they were referencing. Without this when clients would send data, such as their position, receiving clients would not be able to differentiate which client sent the data.

The enemy manager acted similarly to the game manager, spawning enemies and adding then to an enemy dictionary so that when the host sent updates about a certain enemy's position or state, receiving clients could identify the right enemy to update via the dictionary.

4.3.3 Limitations

4.3.3.1 Networking Functions

Functions that required networking had to be programmed manually. This required me to be knowledgeable of net code and networking, meaning the API could not be used by

developers who lack this knowledge. Networked functions had to be manually programmed across all relevant scripts, making the process of adding new functions or changing existing functions extremely lengthy and increased the possibility of user errors occurring. This issue could be remedied by using custom attributes that would manage networking a function for the user, unfortunately this feature was cut due to the time constraints of the project.

4.3.3.2 Contexts Influencing Development

Since the demo was developed alongside the API, this resulted in some features of the API being tailored to the demo game making them less robust and flexible. These features were retroactively changed but they still need testing in multiple different contexts and environments.

4.3.4 Results

The demo game showcased the feasibility of a decentralized topology as it behaves very similarly to a client-server model except one for the clients is the host. When testing the demo with simulated latency, clients retained complete control over their own actions such as moving and shooting, only experiencing delay when receiving feedback for their actions such as destroying an enemy or when other clients were performing actions.

Unexpectedly, the actual delay experienced by clients would far exceed the intended delay which meant receiving data, either as feedback from their actions or other clients' actions, would take an extreme amount of time which would often bring the game world to a standstill for all clients besides the host. This made it difficult to test with simulated latency over extended periods of time. I could not identify what caused this issue however if the issue were fixed and the actual delay matched the intended delay then game world would remain responsive but with a slight delay.

4.3.5 Evaluation

The results demonstrated that the API can be used to develop a multiplayer game, due to the lack of features and the unique bug when simulating latency means the API would need considerably more development and testing before being usable in a professional environment.

The main feature missing from this API would be custom attributes. These would reduce code bloat and allow users to network functions without touching the relevant scripts as users would only need to write this attribute above a function for it to become networked.
Overall, the demo game highlights the difficulty of developing a game using a decentralized topology (RQ 3). This is mainly due to the lack of user centred features, such as custom attributes, as these allow developers to use the API without understanding how it works as well as speed up the development process.

5. Conclusion

The model offers a unique approach to designing multiplayer for games. The Literature Review outlines the various ways in which network latency can affect the user experience, with each section offering different methods and techniques that can be implemented to alleviate symptoms. Both answering RQ 1 and providing solutions to combat the effects of latency.

The analysis of other APIs highlights what features are necessary such as lag compensation and prediction as well as outlining the features of a centralized network topology. Due to the topology of the proposed model, latency can be reduced in some scenarios such as when two or more peers have a lower latency to each other in comparison to their latency to the server. This can help address some of latency's effects on the player experience, but techniques such as lag compensation have the greatest influence on improving the player experience if implemented correctly.

Security is a concern with this model as clients have authority over their own actions and the host peer has all information relevant to simulating the game world which can be used to gain an advantage over others, methods to protect against cheaters such as a referee can be implemented but the game type or genre has a great effect on whether this would work or not. The analysis of existing APIs and the development of my proposed model showcase the differences between centralized and decentralized network topologies (RQ 2).

There were difficulties faced when developing a demo game using a decentralized API. The existence of both a physics and logic host meant two server classes had to be managed, both with their own set of classes (packet, send, receive) that each had to be updated when a new feature was added. Also deciding what game logic was to be handled by the logic server and what could be handled by clients. With logic tied to level progression being managed by the logic server but enemy AI logic such as their current target or path finding was handled by the host client. These difficulties showcase the challenge of developing a game with an atypical network topology (RQ 3).

From the features present in other APIs and the results of developing a demo game using the model, user centred design is a critical element when creating developer tools. Ensuring the API is easy to use as well as adaptable to various game genres and individual scenarios can make or break whether it is picked over others. It is imperative that tool creators consider the end user's needs and wants when developing as more people will interact with their product than they will spend developing it. Future development of the API would be heavily influenced by user-centred design, adding features that are flexible, easy to use and implement.

Bibliography

Bamutange, B., Ramsurrun, V., Seeam, A., Katsina, P., & Anantwar, S. (2020). Zoneless Load Balancing for Massively Multiplayer Online Games. *2020 3rd International Conference on Emerging Trends in Electrical, Electronic and Communications Engineering (ELECOM)* (pp. 173-178). IEEE. doi:10.1109/ELECOM49001.2020.9296989 (Accessed: 28 March 2025)

Beigbeder, T., Coughlan, R., Lusher, C., Plunkett, J., Agu, E., & Claypool, M. (2004). The effects of loss and latency on user performance in unreal tournament 2003®. *NetGames '04: Proceedings of 3rd ACM SIGCOMM workshop on Network and system support for games* (pp. 144–151). New York: Association for Computing Machinery. doi:10.1145/1016540.1016556 (Accessed: 14 April 2025)

Bernier, Y. W. (2003). Latency Compensating Methods in Client/Server In-game Protocol Design and Optimization. Availible at: https://web.cs.wpi.edu/~claypool/courses/4513-B03/papers/games/bernier.pdf (Accessed: 14 April 2025)

Bettner, P., & Terrano, M. (2001). Network Programming in Age. *GDC2001.* Availible at: https://zoo.cs.yale.edu/classes/cs538/readings/papers/terrano_1500arch.pdf (Accessed: 18 February 2025)

Chen, K.-T., Huang, P., Wang, G.-S., Huang, C.-Y., & Lei, C.-L. (2006). On the Sensitivity of Online Game Playing Time to Network QoS. *Proceedings - IEEE INFOCOM*. doi:10.1109/INFOCOM.2006.286 (Accessed: 14 April 2025)

Claypool, M., & Claypool, K. (2006). Latency and player actions in online games. *Commun. ACM, 49*, 40–45. doi:10.1145/1167838.1167860 (Accessed: 14 April 2025)

FirstGearGames. (2022, January). Fish-Networking. Availible at: https://fish-networking.gitbook.io/docs#no-break-promise (Accessed: 14 April 2025)

García Fernández de Castro, E. J., & Jabba Molinares, D. (2023). Literature Review: Latency Compensation Techniques for Online Gaming under an IoT Approach. *2023 IEEE Colombian Caribbean Conference (C3)* (pp. 1-6). Barranquilla: IEEE. doi:10.1109/C358072.2023.10436174 (Accessed: 1 May 2025)

Hami, S. A., Boujelben, Y., & Zarai, F. (2024). Enhancing Cloud Gaming Experience through Optimized Virtual Machine Placement: A Comprehensive Review. *Journal of Network and Systems Management*. doi:10.1007/s10922-024-09864-2 (Accessed: 2 May 2025)

Harcsik, S., Petlund, A., Griwodz, C., & Halvorsen, P. (2007). Latency evaluation of networking mechanisms for game traffic. *Proceedings of the 6th Workshop on Network and System Support for Games, NETGAMES 2007* (pp. 129-134). Melbourne: ACM. doi:10.1145/1326257.1326280 (Accessed: 14 April 2025)

Lee, K.-W., Ko, B.-J., & Calo, S. (2005). Adaptive server selection for large scale interactive online games. *Computer Networks, 49*(1), 84-102. doi:https://doi.org/10.1016/j.comnet.2005.04.006. (Accessed: 14 April 2025)

Madanapalli, S. C., Gharakheili, H. H., & Sivaraman, V. (2022). Know Thy Lag: In-Network Game Detection and Latency Measurement. *Passive and Active Measurement. PAM 2022* (pp. 395–410). Springer. doi:10.1007/978-3-030-98785-5_17 (Accessed: 2 May 2025)

Pellegrino, J. D., & Dovrolis, C. (2003). Bandwidth requirement and state consistency in three multiplayer game architectures. *Proceedings of the 2nd workshop on Network and system support for games* (pp. 52–59). Redwood City: Association for Computing Machinery. doi:https://doi.org/10.1145/963900.963905 (Accessed: 14 April 2025)

Sabet, S. S. (2023). Delay Compensation Technique. In *The Influence of Delay on Cloud Gaming Quality of Experience* (pp. 91–118). Springer, Cham.

doi:https://doi-org.libproxy.abertay.ac.uk/10.1007/978-3-030-99869-1_5 (Accessed: 1 May 2025)

Savery, C., & Graham, T. C. (2013). Timelines: simplifying the programming of lag compensation for the next generation of networked games. *Multimedia Systems*, 271–287. doi:https://doi-org.libproxy.abertay.ac.uk/10.1007/s00530-012-0271-3 (Accessed: 14 April 2025)

Spina, D., Bottino, A., Cavagnino, D., Chiariglione, L., Lucenteforte, M., Mazzaglia, M., & Strada, F. (2024). AI Server-Side Prediction for Latency Mitigation and Cheating Detection: The MPAI-SPG Approach. *2024 IEEE Gaming, Entertainment, and Media Conference (GEM)* (pp. 1-6). Turin: IEEE. doi:10.1109/GEM61861.2024.10585519. (Accessed: 2 May 2025)

Vlahovic, S., Suznjevic, M., & Skorin-Kapov, L. (2019). The Impact of Network Latency on Gaming QoE for an FPS VR Game. *2019 Eleventh International Conference on Quality of Multimedia Experience (QoMEX)* (pp. 1-3). Berlin: IEEE. doi:10.1109/QoMEX.2019.8743193. (Accessed: 14 April 2025)

Ludography

Valve (1998) Half-Life [video game]. Valve.

Available at: https://store.steampowered.com/app/70/HalfLife/ (Accessed: 2 May 2025)

Valve (1999) Team Fortress Classic [video game] Valve.

Available at: https://store.steampowered.com/app/20/Team_Fortress_Classic/ (Accessed: 2 May 2025)

Valve (2000) Counter-Strike [video game] Valve.

Available at: https://store.steampowered.com/app/10/CounterStrike/ (Accessed: 2 May 2025)